Especificação de Requisitos & Roadmap (Atualizado)

Sistema Genérico para Consultório Médico com Cursor IA e Busca de CID

1. Confirmação de Escopo e Novas Funcionalidades

Este documento consolida o escopo definitivo, enxuto e prático para o desenvolvimento do sistema médico através do Cursor IA. Foram eliminados módulos burocráticos externos (como Notas Fiscais, Telemedicina e Estoque) para priorizar a usabilidade interna, segurança de dados locais e estabilidade comercial.

> **📌 Item Confirmado: Armazenamento de Exames e CID-10**
>
> **Armazenamento de Exames:** O sistema gerencia o upload de PDFs ou imagens de exames trazidos pelos pacientes. A secretaria realiza o upload e o médico os acessa diretamente no prontuário.
> **Módulo CID-10 Integrado:** Adicionada a capacidade de busca inteligente e preenchimento automático do CID-10 nas consultas, receitas e atestados médicos.

Recursos Incorporados no Escopo:

- **Controle de Licença por Contra-Senha:** Bloqueio automático do software na data de vencimento. Liberação mediante contra-senha baseada no identificador da clínica e mês de expiração.
- **Cadastro Inicial da Clínica e Logo:** Setup Wizard no primeiro acesso para coletar os dados do consultório e sua logo, aplicando-a nos cabeçalhos de todos os documentos gerados em PDF.
- **Backup e Restore Local:** Botão para compactação zip do banco SQLite (`dados.db`) e da pasta de arquivos anexos (logos e exames) para total portabilidade e proteção contra falhas.

2. Estrutura de Banco de Dados Atualizada (SQLite)

Modelo relacional otimizado para o armazenamento local no formato SQLite:

Tabela	Campos Principais	Finalidade
Clinica	id, nome, cnpj_cpf, telefone, endereco, logo_path	Dados do consultório e logo para impressões.
Config_Licenca	data_instalacao, data_vencimento, ultima_contra_senha	Controle comercial de expiração.
Usuarios	id, nome, login, senha_hash, perfil (medico / secretaria)	Níveis de permissão e segurança.
Pacientes	id, nome, data_nasc, cpf, telefone, email	Dados cadastrais básicos.
Agendamentos	id, paciente_id, data, hora, status (Agendado/ Recepção/Atendimento/Finalizado)	Controle visual do fluxo de espera.
Prontuarios	id, paciente_id, data_atendimento, anamnese, conduta, **cid_codigo, cid_descricao**	Histórico clínico com suporte ao CID-10.
Exames_Anexos	id, paciente_id, data_upload, descricao, arquivo_path	Armazenamento local de exames físicos escaneados.

3. Roteiro de Desenvolvimento em Etapas (Para o Composer do Cursor)

Fase 1: Configuração Estrutural, SQLite e Licenciamento

- **Ação com o Cursor:** Peça à IA para inicializar o banco SQLite com a estrutura acima, a rotina de validação da data de vencimento da licença e a tela de Setup Inicial para cadastro da clínica com upload de logo.

Fase 2: Autenticação de Usuários e Perfis de Acesso

- **Ação com o Cursor:** Criar a tela de login. Redirecionar o usuário "secretaria" estritamente para a visualização da agenda e cadastros, e o usuário "medico" para a fila de atendimento, prontuários e configurações administrativas.

Fase 3: Módulo Recepção (Agenda e Upload de Exames)

- **Ação com o Cursor:** Desenvolver a tela de gerenciamento de horários e a ficha do paciente. Na ficha, implementar o componente de upload de arquivos que salva PDFs/Imagens de exames diretamente em uma pasta local do sistema (ex: `/uploads/exames/`) e salva a referência no banco de dados.

Fase 4: Módulo Médico (Prontuário, Busca de CID e Emissão de Documentos)

- **Ação com o Cursor:** Desenvolver a interface do médico. Ao abrir um atendimento "Na Recepção", renderizar o histórico de consultas e a aba de exames anexados. No formulário de atendimento, implementar um input de autocomplete que consome a rota de CID-10 local do backend. Ao salvar, os dados clínicos e o CID devem estar disponíveis para renderizar a impressão em PDF da receita ou atestado com a logo da clínica.

Fase 5: Ferramentas Administrativas (Backup e Restore)

- **Ação com o Cursor:** Criar as funções de utilitário do sistema para exportar o arquivo `.db` e a pasta de uploads compactados em formato `.zip`, bem como a rotina reversa de restauração de banco de dados.

4. Recomendação Técnico e Dicas de Ouro para o Cursor IA

> **⭐ Recomendação Oficial: Implementation do CID-10 Local via JSON Estático**
>
> A melhor abordagem para um sistema médico genérico e ágil é a **busca local através de um arquivo JSON estático do CID-10 no backend Node.js**, preterindo APIs online da OMS ou bancos de dados inflados. O CID-10 é uma tabela consolidada mundialmente que não sofre mais alterações estruturais. Um arquivo JSON contendo todos os códigos e diagnósticos em português possui cerca de 1.5 MB.
> **Por que esta é a melhor opção?** Ela garante que o sistema funcione com velocidade instantânea e, mais importante, **funciona perfeitamente em cenários onde o consultório fique sem conexão com a internet**. O médico jamais ficará impossibilitado de emitir um atestado com CID por oscilações de rede externa.

💡 1. Como Pedir o Arquivo de CID ao Cursor

Utilize o seguinte prompt: *"Cursor, configure um arquivo chamado `cid10.json` no meu backend Node.js contendo a lista indexada do CID-10 em português. Crie um endpoint de API `GET /api/cid?q=termo` que realize um filtro rápido (pelo código ou descrição) e retorne apenas os 10 primeiros resultados para alimentar o componente de autocomplete do front-end."*

💡 2. Use o Recurso de Contexto Ampliado (@)

Para correções de bugs nas telas, sempre mencione o componente exato usando **@NomeDoArquivo** no Composer (ex: `"Ajuste a exibição dos exames salvos no @ProntuarioMedico.jsx"`). Isso impede que a IA alucine ou modifique partes indesejadas do software.

💡 3. Controle Estrito de Escopo nas Edições

Se o Cursor tentar reescrever arquivos extensos para alterações pontuais, aborte a geração e limite sua ação: `"Modifique estritamente a função de validação da contra-senha dentro do @licencaService.js, mantendo a estrutura original intacta."`

💡 4. Lógica de Geração da Contra-Senha de Licença

Instrua o Cursor a criar a função de cálculo baseada em semente temporal simples para a contra-senha. Por exemplo: concatenar o mês+ano de expiração desejado com uma palavra-chave secreta sua, rodar um hash simples (MD5 ou similar de poucos caracteres) e extrair os 6 primeiros dígitos. Assim, você consegue gerar a contra-senha para o seu cliente sem precisar estar conectado à internet.